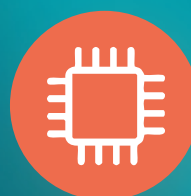
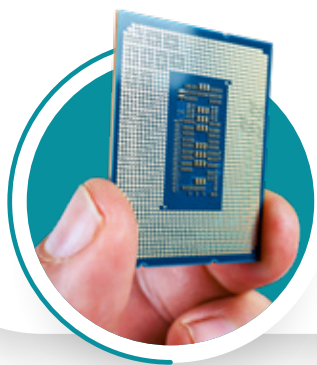




Mantener ocupada la CPU al 100 %





Consideraciones iniciales •

La concurrencia es un concepto primordial en los sistemas operativos modernos. Se entiende como la capacidad de un sistema para ejecutar múltiples procesos o hilos durante el mismo periodo de tiempo, aunque no necesariamente al mismo instante. A diferencia del paralelismo, que implica ejecución de diversas actividades simultáneamente en varios procesadores (un concepto físico que requiere múltiples CPU), la concurrencia se logra mediante la intercalación controlada de ejecución, utilizando técnicas como el cambio de contexto, la organización de procesos y la sincronización. Es un rasgo esencial en los sistemas operativos actuales, puesto que posibilita la realización de varios procesos o hilos en un mismo lapso temporal.

Para nosotros, desarrolladores de *software*, entender e implementar la concurrencia es crucial, ya que es esencial la ejecución paralela para mejorar el desempeño de sistemas operativos actuales, tanto en el ámbito del *software* como en el de los dispositivos físicos. Antes de implementar, es vital asentar las bases teóricas. Esto implica diferenciar entre los distintos tipos de procesamiento y entender los desafíos que la concurrencia introduce.

→ Tipos de procesamiento

La gestión de datos en los sistemas operativos puede llevarse a cabo a través de diversos modelos, en función del diseño del sistema, la estructura del *hardware* y las exigencias para la realización de varias tareas. A continuación, se detallan los tipos de procesamiento más relevantes.

a. Procesamiento secuencial

Es el tipo más básico, en donde las instrucciones o procesos se ejecutan uno tras otro en un único hilo de ejecución. No existe simultaneidad ni paralelismo. Es propio de sistemas monoprogramados y se caracteriza por su simplicidad, aunque limita el aprovechamiento del *hardware* moderno.

- ▶ **Ventaja:** simplicidad en la gestión del sistema.
- ▶ **Desventaja:** bajo rendimiento en tareas que pueden ejecutarse en paralelo.

b. Procesamiento en paralelo

Este modelo permite la ejecución simultánea de múltiples procesos o subprocesos utilizando múltiples unidades de procesamiento (núcleos o procesadores). Cada unidad trabaja en una parte del problema o en tareas diferentes al mismo tiempo.

- ▶ **Aplicación:** procesamiento de imágenes, simulaciones científicas y tareas de alto rendimiento.
- ▶ **Requiere:** sincronización, gestión de concurrencia y control de condiciones críticas.

c. Procesamiento concurrente

Se refiere a la capacidad de un sistema para gestionar múltiples procesos que se ejecutan en “aparente” simultaneidad, aunque no necesariamente al mismo tiempo. Utiliza técnicas como el cambio de contexto, los hilos y la planificación.

- ▶ **Ejemplo:** sistemas multitarea donde múltiples aplicaciones se ejecutan de forma intercalada.
- ▶ **Desafíos:** exclusión mutua, condiciones de carrera y sincronización.

d. Procesamiento distribuido

Consiste en distribuir la ejecución de tareas entre múltiples nodos conectados en red. Cada nodo puede ser un sistema independiente que coopera para alcanzar un objetivo común.

- ▶ **Ejemplo:** computación en la nube, sistemas cliente-servidor y *blockchain*.
- ▶ **Ventajas:** escalabilidad, tolerancia a fallos y rendimiento colaborativo.
- ▶ **Complejidad:** alta, debido a la coordinación, la latencia y la consistencia de datos.

e. Multiprogramación

Permite que varios programas se carguen en memoria y se ejecuten aparentemente al mismo tiempo. El sistema operativo administra los recursos compartidos (CPU y memoria) y alterna su uso entre procesos activos.

- ▶ Fundamental para sistemas interactivos y de tiempo compartido.
- ▶ **Requiere:** algoritmos de planificación de procesos y control de acceso a recursos.

f. Procesamiento multinúcleo

Es un tipo de procesamiento paralelo que se lleva a cabo específicamente en arquitecturas con múltiples núcleos en un solo chip. Cada núcleo puede ejecutar procesos de forma independiente.

- ▶ **Ejemplo:** computadoras modernas con CPU de 4, 8 o más núcleos.
- ▶ **Optimización:** es crucial el uso de hilos y técnicas de paralelismo eficiente.



Desafíos al manejar la concurrencia

Los sistemas operativos enfrentan desafíos significativos al manejar la concurrencia, como las condiciones de carrera y los interbloqueos. Una condición de carrera ocurre cuando la salida de un programa depende del orden de ejecución de operaciones concurrentes, lo que puede llevar a resultados incorrectos. Por su parte, un interbloqueo (deadlock) es una condición no deseada en los sistemas operativos concurrentes, en donde dos o más procesos se encuentran bloqueados permanentemente por estar esperando recursos que están siendo retenidos por otros procesos. En otras palabras,

cada proceso espera indefinidamente a que otro libere un recurso, lo que genera un ciclo sin salida. Un interbloqueo se produce únicamente si se cumplen simultáneamente las siguientes cuatro condiciones:

¿Se cumplen las condiciones para un interbloqueo?



El manejo adecuado del interbloqueo es esencial para **garantizar la eficiencia del sistema**, la continuidad del servicio y la seguridad e integridad de los datos y procesos concurrentes.

Dentro de este contexto, surgen los semáforos, que son mecanismos de sincronización utilizados en sistemas operativos para controlar el acceso concurrente a recursos compartidos por múltiples procesos o hilos. Fueron introducidos por Edsger Dijkstra, en 1965, y constituyen una herramienta fundamental para evitar condiciones de carrera y garantizar la exclusión mutua.

Existen diferentes tipos de semáforos. El más reconocido es el semáforo binario (*mutex*), que solo tiene dos valores posibles (0 o 1). Se utiliza para exclusión mutua. Otro es el semáforo contador, que permite que múltiples procesos accedan a una sección crítica en función de la disponibilidad del recurso.

Por otro lado, se pueden realizar operaciones básicas. La operación `wait(S)` decrementa el valor del semáforo; si el resultado es menor que cero, el proceso se bloquea. En cambio, `signal(S)` incrementa el valor del semáforo y, si hay procesos bloqueados, se despierta uno de ellos. Entre sus aplicaciones se encuentra el control de acceso a secciones críticas, la sincronización entre productor-consumidor y la implementación de monitores o barreras.

Asimismo, los mensajes son un método de comunicación entre procesos (IPC) que facilita a los procesos el intercambio de información sin la necesidad de compartir memoria. Esta técnica se divide principalmente en dos categorías: la comunicación sincrónica, en donde el emisor se interrumpe hasta que el receptor recibe el mensaje; y la comunicación asincrónica, en la que el emisor prosigue con su ejecución justo después de transmitir el mensaje. Las dos modalidades proporcionan soluciones eficaces dependiendo del grado de sincronización necesario entre los procesos.

Por último, las tuberías (*pipes*) permiten una comunicación unidireccional entre procesos, los sockets facilitan la conexión tanto entre procesos locales como distribuidos a través de redes y las colas de mensajes gestionan los datos siguiendo un orden FIFO o basado en prioridades. Esta técnica ofrece ventajas importantes, como el aislamiento entre procesos, un mayor control sobre los mecanismos de sincronización y una alta escalabilidad, especialmente en sistemas distribuidos o en arquitecturas orientadas a servicios.



→ Concurrency

La concurrencia se refiere a la capacidad de un sistema para ejecutar múltiples procesos o hilos durante el mismo periodo de tiempo, aunque no necesariamente al mismo instante. A diferencia del paralelismo, la concurrencia no implica simultaneidad física, sino la intercalación controlada de ejecución.

La concurrencia es un rasgo esencial en los sistemas operativos actuales, puesto que posibilita la realización de varios procesos o hilos en un mismo lapso temporal. Para alcanzar este objetivo, se emplean métodos como el cambio de contexto, la organización de procesos y la sincronización, que garantizan un uso ordenado y eficaz de los recursos del sistema. La concurrencia se implementa tanto en procesos autónomos como en hilos dentro de un proceso común (*multithreading*), lo que permite un uso más eficiente de las capacidades del equipo.

No obstante, la concurrencia también plantea retos significativos. Entre las dificultades más habituales se hallan las condiciones de carrera, los interbloqueos y las infracciones de exclusión mutua, que pueden perjudicar la estabilidad y fiabilidad del sistema si no se manejan correctamente.

A pesar de estos riesgos, su implementación aporta beneficios clave como el aumento de la eficiencia del CPU, la posibilidad de ejecutar múltiples tareas de forma interactiva y la base estructural para el diseño de sistemas multitarea y multiproceso.

→ Escenario problematizado: el SSSM de Medellín

Para afianzar nuestra comprensión y prepararnos para los desafíos del mundo real en el desarrollo de *software*, proponemos un escenario práctico que, si bien es hipotético, refleja la complejidad de los sistemas empresariales modernos.

La Asociación de Gestión Empresarial y Tecnología contactó a un equipo de desarrolladores para lograr un sistema de soporte de servicios públicos en Medellín (SSSM), vital para su operación diaria. Este SSSM no es una aplicación monolítica; está compuesto por múltiples módulos que deben operar de manera concurrente para ser eficientes y responsivos.

- ▶ **Módulo de gestión de pedidos (ventas):** miles de usuarios intentan simultáneamente registrar órdenes de nuevos servicios o modificaciones de planes.
- ▶ **Módulo de inventario y recursos (almacenes):** se encarga de la asignación y el seguimiento de equipos (*modems*, cables, etc.) que se instalan o retiran en campo.
- ▶ **Módulo de facturación y cobranzas (finanzas):** genera facturas en tiempo real y procesa pagos, accediendo a los mismos datos de pedidos y servicios.
- ▶ **Módulo de soporte técnico (atención al cliente):** múltiples agentes acceden a historiales de clientes y estados de servicio para resolver incidencias.

→ Problema

¿Qué mecanismos utiliza el SSSM para evitar que múltiples procesos accedan al mismo recurso al mismo tiempo sin generar errores o pérdida de datos? ¿Cómo se logra esta coordinación en un entorno de procesamiento paralelo? ¿Qué pasaría si, al intentar abrir todas las aplicaciones al mismo tiempo, el sistema operativo no decidiera qué se ejecuta, cuándo y cómo? ¿Podríamos usarlo de forma fluida o segura?

→ Aplicación de la **conurrencia** en el SSSM

En un sistema como el SSSM, cada acción del usuario o cada proceso automatizado se traduce en la creación y gestión de hilos o procesos concurrentes.

a. **Conurrencia en la gestión de pedidos:**

Cuando un agente de ventas registra una nueva orden, se inicia un hilo (o proceso) específico para manejar ese pedido. Si diez agentes registran pedidos al mismo tiempo, el sistema debe ser capaz de ejecutar diez hilos concurrentemente. Cada hilo interactuará con la base de datos de clientes, el inventario de equipos y el módulo de facturación.

b. Necesidad de sincronización (exclusión mutua):

la sincronización se vuelve crítica.

- ▶ **Actualización de inventario:** si el Hilo A (Pedido 1) y el Hilo B (Pedido 2) intentan reservar el último *modem* disponible en el inventario al mismo tiempo, se produciría una condición de carrera. Ambos podrían verificar que hay un *modem*, intentar reservarlo, y generar un error lógico o una doble asignación. Para evitarlo, se usaría un semáforo binario o mutex. Antes de acceder al registro del *modem* en la base de datos, el hilo intentaría adquirir el mutex. Solo uno lo conseguiría; el otro esperaría. Una vez que el *modem* es reservado y el inventario actualizado, el mutex es liberado. En C++, esto se implementaría con `std::mutex` o `std::lock_guard`.
- ▶ **Procesamiento de pagos:** múltiples hilos del módulo de facturación podrían intentar actualizar el saldo de un mismo cliente simultáneamente. Un mutex sobre el registro del cliente garantizaría la integridad del saldo, lo que asegura que las operaciones se realicen una a una.

c. Comunicación entre procesos (IPC) y sincronización avanzada:

- ▶ **Sincronización productor-consumidor:** el módulo de pedidos podría actuar como productor de eventos de “nuevo pedido” y el módulo de inventario como consumidor de estos eventos. Se podría utilizar una cola de mensajes (implementada con semáforos o monitores) para desacoplar estos procesos. El módulo de pedidos añade mensajes a la cola, y el módulo de inventario los consume cuando están disponibles. Si la cola está vacía, el consumidor (inventario) espera; si está llena, el productor (pedidos) espera. Esto optimiza el flujo de trabajo y evita que un módulo sobrecargue al otro.
- ▶ **Paso de mensajes (sockets/RPC):** para la comunicación entre módulos distribuidos (por ejemplo, si el módulo de Facturación está en un servidor diferente al de Inventario), se usarían mecanismos de “comunicación sincrónica” o “asincrónica”, como los sockets. Esto permite a los módulos interactuar de forma aislada, intercambiando información sin necesidad de compartir memoria directamente.

d. Prevención de interbloqueos (*deadlocks*):

En el SSSM, un interbloqueo podría ocurrir si el Hilo A necesita el Recurso X (un registro de cliente) y luego el Recurso Y (un registro de inventario), mientras que el Hilo B necesita el Recurso Y y luego el Recurso X. Si ambos hilos adquieren su primer recurso simultáneamente e intentan adquirir el segundo, se bloquearían mutuamente. Para prevenir esto, se impone un orden estricto en la adquisición de recursos: todos los hilos deben adquirir los recursos en el mismo orden (por ejemplo, siempre primero el registro de cliente, luego el de inventario), lo que rompe la condición de “espera circular”.

→ Este escenario del SSSM transforma la teoría en un desafío de ingeniería

- ▶ **Identificación de concurrencia:** reconocer cuándo múltiples hilos son necesarios para mejorar el rendimiento y la responsividad del sistema.
- ▶ **Diseño de soluciones robustas:** diseñar algoritmos y estructuras de datos que sean thread-safe; es decir, que funcionen correctamente en un entorno multihilo.
- ▶ **Elección de mecanismos de sincronización:** decidir cuándo usar *mutexes*, cuándo semáforos contadores, cuándo colas de mensajes o cuándo otras primitivas de sincronización. Por ejemplo, en el caso de la “cena de los filósofos”, las soluciones son usar semáforos para controlar el acceso a los tenedores o permitir que solo cuatro filósofos se sienten a la mesa al mismo tiempo, principios aplicables a la gestión de recursos limitados en cualquier sistema.
- ▶ **Análisis y depuración:** aprender a identificar y resolver problemas de concurrencia (condiciones de carrera, interbloqueos, inanición). Estos errores son difíciles de reproducir y depurar, por lo que requiere un pensamiento lógico y sistemático.
- ▶ **Optimización de recursos:** entender cómo la concurrencia contribuye a la eficacia (mantener ocupada la CPU al 100 % y maximizar el número de tareas procesadas).



Créditos

Expertos temáticos: Juan Camilo David Díaz y Julián Galeano Echeverri
Diseño de la experiencia virtual de aprendizaje: Sebastián Higuita Hernández
Validación pedagógica: Jhobana Herrera Díaz
Corrección de estilo: Reis Ríos
Diseño gráfico: Alejandra Ochoa Mesa



INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO®
Acreditados en Alta Calidad

